

Chapter 1

HEURISTIC APPROACHES TO PRODUCTION- INVENTORY-DISTRIBUTION PROBLEMS IN SUPPLY CHAINS

Burak Eksioglu

*Department of Industrial and Systems Engineering,
University of Florida,
303 Weil Hall, P.O. Box 116595,
Gainesville, Florida 32611-6595
burak@grove.ufl.edu*

Athanasios Migdalas

*Department of Production Engineering and Management,
Technical University of Crete,
73100 Chania, Greece
migdalas@ergasya.tuc.gr*

Panos M. Pardalos

*Department of Industrial and Systems Engineering,
University of Florida,
303 Weil Hall, P.O. Box 116595,
Gainesville, Florida 32611-6595
pardalos@cao.ise.ufl.edu*

Abstract In this paper we focus on global optimization issues in supply chains. We propose solution methods for production, inventory and distribution problems with fixed charge costs that arise in a supply chain. The production-inventory-distribution problem with fixed charges falls under the category of minimum concave cost network flow problems which are known to be NP-hard. We present two heuristic approaches; a dynamic slope scaling procedure (DSSP) and a greedy randomized adaptive search procedure (GRASP). DSSP and GRASP both have strong

intuitive appeal and are straightforward to implement. We compare the two heuristic approaches by presenting a broad set of computational results.

Keywords: Concave Network Flow Problems, Fixed Charge Costs, Computational Experiments, Branch and Bound, Mixed Integer Programming, Supply Chain.

1. Introduction

The term supply chain management (SCM) has been around for more than twenty years. Its definition varies from one enterprise to another. We define a supply chain (SC) as an integrated process where different business entities such as suppliers, manufacturers, distributors, and retailers work together to plan, coordinate, and control the flow of materials, parts, and finished goods from suppliers to customers. This chain is concerned with two distinct flows: a forward flow of materials and a backward flow of information. A recent review on supply chain management models and applications can be found in [8].

In this paper we focus on global optimization issues in a SC among a set of manufacturers and a set of retailers. In particular, we consider a problem where the goal is to meet the known demand at the retailers through production at the plants such that the system wide total production, inventory, and distribution cost is minimized. This problem can be formulated as a network flow problem on a directed, single source, graph consisting of several layers. Figure 1.1 gives an example with two plants, three retailers and two time periods. Each layer of the graph represents a time period. In each layer, a bipartite graph represents the transportation network between the plants and the retailers. Plants in successive time periods are connected through inventory arcs. There is a dummy source node with supply equal to the total demand. Production arcs connect the dummy source to each plant in every time period.

This is an easy problem if all costs are linear. However, many production and distribution activities exhibit economies of scale in which the unit cost of the activity decreases as the volume of the activity increases. For example, production costs often exhibit economies of scale due to fixed production setup costs and learning effects that enable more efficient production as the volume increases. Transportation costs exhibit economies of scale due to the fixed cost of initiating a shipment and the lower per unit shipping cost as the volume delivered per shipment increases. Therefore, we assume the production cost at the plants and the cost of transporting goods from plants to retailers are all of the

fixed charge type, whereas inventory costs are linear. We also make the following simplifying assumptions to the model:

- Backorders are not allowed.
- Products are stored at their production location until being transported to a retailer.
- There are no capacity constraints on the production, inventory, or distribution arcs.
- Transportation is not allowed between plants.

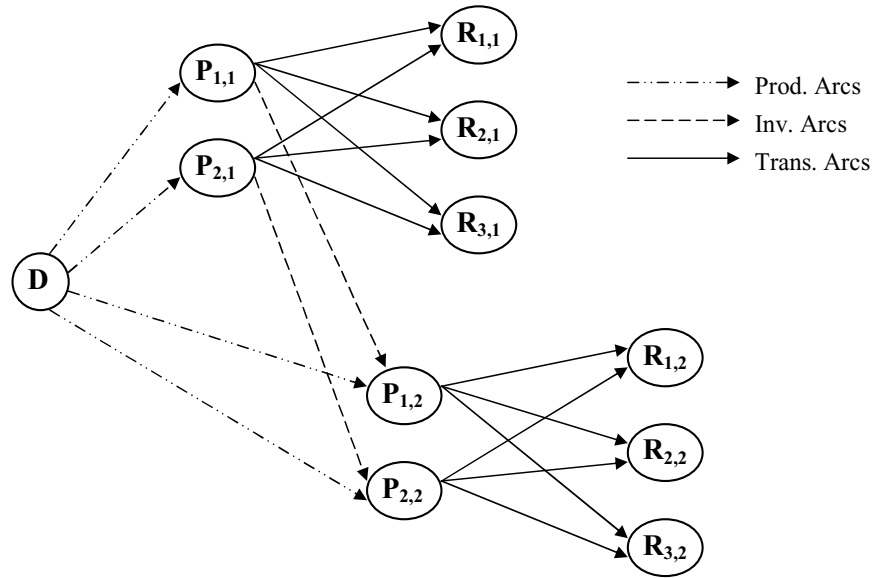


Figure 1.1. A supply chain network with two plants, three retailers, and two periods.

Freling et al. [7] and Romeijn and Romero Morales [26, 27, 28, 29] consider similar problems. They assume that production and inventory costs are linear and that there is a fixed cost of assigning a plant to a retailer. They also account for the presence of so-called single-sourcing constraints where each retailer should be supplied from a single plant only. Eksioglu et al. [5] and Wu and Golbasi [36] consider the multi commodity case where there are multiple products flowing on the network. Eksioglu et al. [5] assume production and inventory costs are linear and transportation costs are of fixed charge type, whereas Wu and Golbasi [36] assume production costs are fixed charge and inventory and transportation costs are linear.

The production-inventory-distribution problem with fixed charge costs falls under the category of minimum concave cost network flow problems (MCCNFP). Guisewite and Pardalos [10] give a detailed survey on MCCNFP throughout the early 1990s. It is well known that even certain special cases of MCCNFP, such as the fixed charge network flow problem (FCNFP) or the single source uncapacitated minimum concave cost network flow problem (SSU MCCNFP), are *NP*-hard [11]. MCCNFP is *NP*-hard even when the arc costs are constant, the underlying network is bipartite, or the ratio of the fixed charge to the linear charge for all arcs is constant. This has motivated the consideration of additional structures which might make the problem more tractable. In fact, polynomial time algorithms have been developed for a number of specially structured variants of MCCNFP [3, 25].

The number of source nodes and the number of arcs with nonlinear costs affect the difficulty of MCCNFP. It is therefore convenient to refer to MCCNFP with a fixed number, h , of sources and fixed number, k , of arcs with nonlinear costs as $\text{MCCNFP}(h, k)$. Guisewite and Pardalos [12] were the first to prove the polynomial solvability of $\text{MCCNFP}(1, 1)$. Later, strongly polynomial time algorithms were developed for $\text{MCCNFP}(1, 1)$ in [21] and [33]. In a series of papers by Tuy et al. [33, 34, 35] and Tuy [31, 32] polynomial time algorithms are presented for $\text{MCCNFP}(h, k)$ and it is also shown that $\text{MCCNFP}(h, k)$ with $\min\{h, k\}=1$ can be solved in strongly polynomial time.

Most of the exact solution approaches transform the problem into an equivalent 0 – 1 Mixed Integer Linear Program (MILP) and use branch and bound techniques. Two of the latest branch and bound algorithms for fixed charge transportation problems are presented by McKeown and Ragsdale [24] and Lamar and Wallace [22]. Recently, Bell et al. [1] presented a branch and bound algorithm for the capacitated fixed charge transportation problem. Most of the branch and bound algorithms use conditional penalties called *up* and *down* penalties, that contribute to finding good lower bounds. Other exact solution algorithms are vertex enumeration techniques and dynamic programming approaches. The computational time required by exact solution approaches grows exponentially as the problem size increases. Our computational experiments indicate that even for problems with only 30 supply points, 70 demand points and 20 time periods the exact approaches fail to find a solution.

The *NP*-hardness of the problem motivates the use of approximate approaches. Since the production-inventory-distribution problem with fixed charge costs falls under the category of MCCNFP, it achieves its optimal solution at an extreme point of the feasible region [15]. Most of the approximate solution approaches exploit this property. Some recent

heuristic procedures have been developed by Holmqvist et al. [14], Diaby [2], Khang and Fujuwara [17], Larsson et al. [23], Ghannadan et al. [9], and Sun et al. [30]. Recently, Kim and Pardalos [18, 19, 20] provide a dynamic slope scaling procedure (DSSP) to solve FCNFP. Eksioglu et al. [4] refine the DSSP and present results for bipartite and layered fixed charge network flow problems.

In the following sections we formulate our problem, describe the two heuristic approaches, and then present computational results. The problem definition and formulation are given in section 2. A detailed description of the DSSP algorithm and the GRASP approach developed for our problem are covered in section 3. The results of our computational study are presented in section 4, and finally section 5 concludes the paper.

2. Problem Definition and Formulation

We consider a multi-facility production, inventory, and distribution problem where a single item is to be produced in multiple plants over multiple periods to satisfy the demand at the retailers such that the system wide total production, inventory, and transportation cost is minimized.

Let J , K , and T denote the number of plants, the number of retailers, and the planning horizon, respectively. Then the resulting model is:

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^J \sum_{t=1}^T r_{jt}(p_{jt}) + \sum_{j=1}^J \sum_{k=1}^K \sum_{t=1}^T f_{jkt}(x_{jkt}) + \sum_{j=1}^J \sum_{t=1}^T h_{jt}I_{jt} \\ \text{subject to} \quad & \end{aligned} \tag{P1}$$

$$p_{jt} + I_{j,t-1} - I_{jt} - \sum_{k=1}^K x_{jkt} = 0 \quad \begin{matrix} j = 1, \dots, J, \\ t = 1, \dots, T, \end{matrix} \tag{1}$$

$$\sum_{j=1}^J x_{jkt} = d_{kt} \quad \begin{matrix} k = 1, \dots, K, \\ t = 1, \dots, T, \end{matrix} \tag{2}$$

$$p_{jt} \leq C_{jt} \quad \begin{matrix} j = 1, \dots, J, \\ t = 1, \dots, T, \end{matrix} \tag{3}$$

$$I_{jt} \leq V_{jt} \quad \begin{matrix} j = 1, \dots, J, \\ t = 1, \dots, T, \end{matrix} \tag{4}$$

$$x_{jkt} \leq U_{jkt} \quad \begin{matrix} j = 1, \dots, J, \\ k = 1, \dots, K, \\ t = 1, \dots, T, \end{matrix} \tag{5}$$

$$p_{jt}, I_{jt}, x_{jkt} \geq 0 \quad \begin{matrix} j = 1, \dots, J, \\ k = 1, \dots, K, \\ t = 1, \dots, T, \end{matrix} \tag{6}$$

where

- p_{jt} = amount produced at plant j in period t .
- x_{jkt} = amount shipped from plant j to retailer k in period t .
- I_{jt} = inventory carried at plant j during period t .
- C_{jt} = production capacity at plant j in period t .
- U_{jkt} = capacity on the shipment from plant j to retailer k in period t .
- V_{jt} = inventory capacity at plant j in period t .
- $r_{jt}(p_{jt})$ = cost of producing p_{jt} units at plant j in period t .
- $f_{jkt}(x_{jkt})$ = cost of shipping x_{jkt} units from plant j to retailer k in period t .
- h_{jt} = unit holding cost per period at plant j for period t .
- d_{kt} = demand at retailer k in period t .

Constraints (1) and (2) are the flow conservation constraints and (3), (4), and (5) are the capacity constraints. If C_{jt} , U_{jkt} , and V_{jt} are large enough then the problem is effectively uncapacitated. As we pointed out earlier MCCNFP achieves its optimal solution at an extreme point of the feasible region, and for the uncapacitated problem this corresponds to a tree solution. In other words, an optimal solution exists in which the demand of each retailer will be satisfied through only one of the plants. Here we consider the uncapacitated problem.

The production cost function $r_{jt}(p_{jt})$ is of the following form:

$$r_{jt}(p_{jt}) = \begin{cases} 0 & \text{if } p_{jt} = 0, \\ s_{jt} + c_{jt}p_{jt} & \text{if } 0 < p_{jt} \leq C_{jt}. \end{cases}$$

where s_{jt} and c_{jt} are, respectively, the setup and the variable costs of production. The distribution cost function, $f_{jkt}(x_{jkt})$, is also of a fixed charge form and is given by:

$$f_{jkt}(x_{jkt}) = \begin{cases} 0 & \text{if } x_{jkt} = 0, \\ s_{jkt} + c_{jkt}x_{jkt} & \text{if } 0 < x_{jkt} \leq U_{jkt}. \end{cases}$$

where s_{jkt} and c_{jkt} represent, respectively, the setup and the variable costs of distribution.

Due to the discontinuity of the cost functions, the problem can be transformed into a 0–1 mixed integer linear programming (MILP) problem by introducing a binary variable for each arc with a fixed charge. Assuming $s_{jt} > 0$, the cost function $r_{jt}(p_{jt})$, can be replaced with:

$$r_{jt}(p_{jt}) = c_{jt}p_{jt} + s_{jt}y_{jt}.$$

Similarly, the distribution cost functions can be replaced with:

$$f_{jkt}(x_{jkt}) = c_{jkt}x_{jkt} + s_{jkt}y_{jkt}.$$

where

$$y_{jt} = \begin{cases} 0 & \text{if } p_{jt} = 0, \\ 1 & \text{if } p_{jt} > 0, \end{cases} \quad \text{and} \quad y_{jkt} = \begin{cases} 0 & \text{if } x_{jkt} = 0, \\ 1 & \text{if } x_{jkt} > 0. \end{cases}$$

The MILP formulation of the problem is as follows:

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^J \sum_{t=1}^T (c_{jt}p_{jt} + s_{jt}y_{jt} + h_{jt}I_{jt}) + \sum_{j=1}^J \sum_{k=1}^K \sum_{t=1}^T (c_{jkt}x_{jkt} + s_{jkt}y_{jkt}) \\ \text{subject to} \quad & \end{aligned} \tag{P2}$$

(1), (2), (4), (6), and

$$p_{jt} \leq C_{jt}y_{jt} \quad \begin{matrix} j = 1, \dots, J, \\ t = 1, \dots, T, \end{matrix} \tag{7}$$

$$x_{jkt} \leq U_{jkt}y_{jkt} \quad \begin{matrix} j = 1, \dots, J, \\ k = 1, \dots, K, \\ t = 1, \dots, T, \end{matrix} \tag{8}$$

$$y_{jt}, y_{jkt} \in \{0, 1\} \quad \begin{matrix} j = 1, \dots, J, \\ k = 1, \dots, K, \\ t = 1, \dots, T. \end{matrix} \tag{9}$$

The MILP formulation is used to find an optimal solution. This formulation is useful in practice since the relaxed problems are linear cost network flow problems. We use the general purpose solver CPLEX [16], which employs branch and bound algorithms, to solve the MILP formulation. This is done to see the difficulty of finding an optimal solution.

3. Solution Procedures

The DSSP and the GRASP algorithms first find an initial feasible solution to the problem, and then improve this solution by a local search procedure. The final solution is compared to a lower bound. Section 3.1 summarizes the DSSP algorithm, Section 3.2 explains the GRASP approach, and Section 3.3 gives details on how lower bounds are obtained.

3.1. The DSSP

The DSSP is a procedure that iteratively approximates the fixed charge cost function by a linear function, and solves the corresponding network flow problem. Note that each of the approximating network problems has exactly the same set of constraints, and differs only with respect to the objective function coefficients. The motivation behind the DSSP is the fact that a concave function, when minimized over a set of linear constraints, will have an extreme point optimal solution. Therefore, there exists a linear cost function that yields the same optimal solution as the concave cost function. The following summarizes the DSSP algorithm:

function DSSP

$q := 0$ /*set the iteration counter to zero*/

Initialize $\bar{c}_{jt}^{(q)}$, $j = 1, \dots, J, t = 1, \dots, T$ /*find an initial linear cost*/

while (stopping criterion not satisfied) **do**

Solve the following network flow problem:

$$\text{minimize } \sum_{j=1}^J \sum_{t=1}^T (\bar{c}_{jt}^{(q)} p_{jt} + h_{jt} I_{jt}) + \sum_{j=1}^J \sum_{k=1}^K \sum_{t=1}^T \bar{c}_{jkt}^{(q)} x_{jkt}$$

subject to (1), (2), (3), (4), (5), and (6).

$q := q+1$

Update $\bar{c}_{jt}^{(q)}$, $j = 1, \dots, J, t = 1, \dots, T$

Perform local search on the current solution, $S^{(q)} = (p^{(q)}, I^{(q)}, x^{(q)})$

if $\text{cost}(S^{(q)}) < \text{cost}(S^*)$ **then** $S^* := S^{(q)}$

end while

return S^*

end function

Algorithm 1. The DSSP algorithm

The local search is the same local search used in the GRASP algorithm, which is given in the following section. Important issues regarding the DSSP algorithm are:

- Finding an initial linear cost ($\bar{c}_{jt}^0$).
- Updating scheme for $\bar{c}_{jt}^{(q)}$.
- Stopping condition.

Kim and Pardalos [18] investigate two different ways to initiate the algorithm. The initiation scheme presented here is shown to provide better results when used for large scale problems (see Figure 1.2). The initial value of the linear cost we use is: $\bar{c}_{jt}^0 = c_{jt} + s_{jt}/C_{jt}$. This is simply the LP relaxation of the original problem. However, note that the problems we are solving are NFPs which can be solved much faster than LPs.

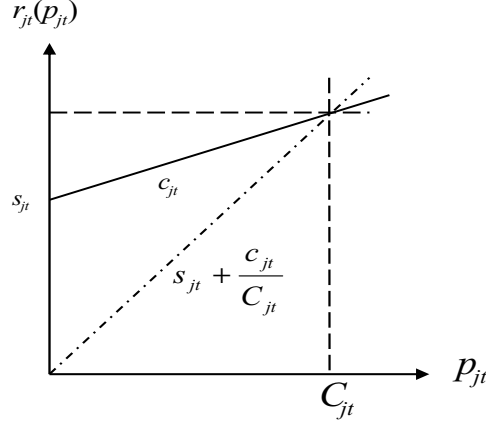


Figure 1.2. Linearization of the fixed charge cost function

The linear cost coefficients are updated in such a way that they reflect the variable costs and the fixed costs simultaneously. Given a feasible solution $S^{(q)} = (p^{(q)}, I^{(q)}, x^{(q)})$ in iteration q , the objective function coefficients for the next iteration are expressed in linear form as:

$$\bar{c}_{jt}^{(q+1)} = \begin{cases} c_{jt} + s_{jt}/p_{jt}^{(q)} & \text{if } p_{jt}^{(q)} > 0, \\ \bar{c}_{jt}^{(q)} & \text{if } p_{jt}^{(q)} = 0, \end{cases}$$

and

$$\bar{c}_{jkt}^{(q+1)} = \begin{cases} c_{jkt} + s_{jkt}/x_{jkt}^{(q)} & \text{if } x_{jkt}^{(q)} > 0, \\ \bar{c}_{jkt}^{(q)} & \text{if } x_{jkt}^{(q)} = 0. \end{cases}$$

If two consecutive solutions in the above algorithm are equal, then the linear cost coefficients and the objective function values in the following iterations will be identical. As a result, once $S^{(q)} = S^{(q-1)}$ there can be no more improvement. Therefore, a natural stopping criterion is to terminate the algorithm when $\|S^{(q)} - S^{(q-1)}\| < \varepsilon$. An alternative is to terminate the algorithm after a fixed number of iterations if the above criterion is not satisfied.

3.2. GRASP

The Greedy Randomized Adaptive Search Procedure [6] is an iterative process which provides a feasible solution within every iteration for combinatorial optimization problems. Each iteration consists of a construction phase and a local search phase. In the construction phase, a randomized greedy function is used to build up an initial solution. This solution is then used for improvement attempts in the local search phase. This iterative process is performed for a fixed number of iterations and the final result is simply the best solution found over all iterations. The number of iterations is one of the two parameters to be tuned. The other parameter is the size of the candidate list in the construction phase.

Construction Phase. In the construction phase a feasible solution is constructed step by step utilizing some of the problem specific properties. Since our problems are uncapacitated concave cost network flow problems the optimal solution will be a tree on the network. Therefore, we construct solutions where each retailer is supplied by a single plant. The construction phase starts by connecting one of the retailers to one of the plants. The procedure finds the plants that give the lowest per unit production and transportation cost for a retailer, taking into account the effect that already connected retailers have on the solution. The unit cost, λ_j , of assigning plant j to retailer k is $c_{jkt} + s_{jkt}/d_{kt} + c_{jt} + s_{jt}/(p_{jt} + d_{kt})$.

The cheapest connections for this retailer are then put into a restricted candidate list, RCL, and one of the plants from RCL is selected randomly. The size of the RCL is one of the parameters that requires tuning. Instead of using a fixed number, we add a plant to the list if its cost is no more than a multiple of the cheapest one. Finally, when the chosen plant is connected to the retailer, the flows on the arcs are updated accordingly. The procedure is given in Algorithm 2. Holmqvist et al. [14] develop a similar GRASP for SSU MCCNFP. Our approach differs from theirs in the greedy function used to initialize the RCL. They use the actual cost where as we check the unit cost of connecting retailers to one of the plants.

procedure Construct

for ($t = 1, \dots, T$) **do**

$p_{jt} := 0, j = 1, \dots, J$

$x_{jkt} := 0, j = 1, \dots, J, k = 1, \dots, K$

for ($k = 1, \dots, K$) **do**

$\lambda_j := c_{jkt} + s_{jkt}/d_{kt} + c_{jt} + s_{jt}/(p_{jt} + d_{kt}), j = 1, \dots, J$

```

 $\Lambda := \min\{\lambda_j : j = 1, \dots, J\}$ 
 $\text{RCL} := \{j : \lambda_j \leq \frac{\Lambda}{\alpha}, \leq < \alpha \leq 1\}$ 
Select  $l$  at random from RCL
 $p_{lt} := p_{lt} + d_{kt}$ 
 $x_{lkt} := d_{kt}$ 
end for  $k$ 
end for  $t$ 
return the current solution,  $S$ 
end procedure

```

Algorithm 2. The construction procedure

When α , in Algorithm 2, is zero the procedure is totally randomized and the greedy function value, λ_j , is irrelevant. However, when α is equal to 1 the procedure is a pure greedy heuristic without any randomization. Best results were obtained for $0.8 \leq \alpha \leq 0.85$ after fine tuning.

The construction procedure in Algorithm 2 handles each period independently. In other words, the initial solution created does not carry any inventory and demands are met through production within the same period. A second approach is to allow demand to be met by production from previous periods. The second approach leads to better initial solutions, but the final solutions after local search are worse compared to the ones we get from the first approach. This is due to the fact that the initial solutions in the second approach are likely already local optima. In the first approach however, we do not start with a very good solution but the local search procedure improves the solution.

Local Search. A local search is applied to the feasible solutions obtained from the DSSP algorithm and the construction phase of the GRASP algorithm in an attempt to improve the solutions. A solution S' is a neighbor to the current feasible solution S if the plant supplying one of the retailers is changed. Note that changing the plant that supplies a retailer may not necessarily result in an extreme point feasible solution. After the flow to a retailer is re-routed through another plant we need to force the solution to be a tree. This procedure is explained in Algorithm 3. The best solution overall is kept as the result.

Let j' be the plant that currently supplies retailer k 's demand in period t through production in period τ' ($t \geq \tau'$). Also, let $\delta_{j\tau}$ be the additional cost of changing the supplier to plant j in period τ . Note that $\delta_{j\tau} = 0$ when $j = j'$ and $\tau = \tau'$. The local search procedure is given in Algorithm 3, where j^* and τ^* are such that $\delta_{j^*\tau^*} = \Delta$.

procedure Local

```

  Let  $S$  be an extreme feasible solution
  while ( $S$  is not a local optimum) do
    for ( $t = 1, \dots, T$ ) do
      for ( $k = 1, \dots, K$ ) do
        Calculate  $\delta_{j\tau}$ ,  $j = 1, \dots, J$ ,  $\tau = 1, \dots, t$ 
         $\Delta := \min\{\delta_{j\tau} : j = 1, \dots, J, \tau = 1, \dots, t\}$ 
        if  $\Delta=0$  then  $S$  is a local optimum
        else if  $\Delta < 0$  then
          Subtract the demand ( $d_{kt}$ ) from its current path
          And flow it through  $j^*$  and  $\tau^*$ 
          if the resulting flow is not a tree then
            Identify all cycles and remove them
            (This can be done by a simple check of the plants
             in each period. If there is production and inventory
             remove one or the other)
          end if
        end if
      end for  $k$ 
    end for  $t$ 
  end while
  return  $S$ 
end procedure

```

Algorithm 3. The local search procedure

3.3. Lower Bounds

The DSSP and GRASP give upper bounds to (P2). These upper bounds are compared to a lower bound to measure the quality of the solutions. Unfortunately, the LP relaxation of (P2) gives poor lower bounds, particularly for problems with large fixed charges. Therefore, we reformulate the problem, which leads to an increase in problem size but gives tighter lower bounds. In the reformulation given below the flows are decomposed by destination. Let $x_{j\tau kt}$ denote the amount produced at plant j in period τ to meet the demand of retailer k in period t . We can now rewrite x_{jkt} and I_{jt} in terms of $x_{j\tau kt}$ as:

$$x_{jkt} = \sum_{\tau=1}^t x_{j\tau kt} \quad \text{and} \quad I_{jt} = \sum_{k=1}^K \sum_{\tau=1}^t \sum_{l=t+1}^T x_{j\tau kl}$$

Substituting these into (P2) will give the following extended formulation.

$$\begin{aligned}
\text{minimize } & \sum_{j=1}^J \sum_{t=1}^T (c_{jt} p_{jt} + s_{jt} y_{jt}) + \sum_{j=1}^J \sum_{k=1}^K \sum_{t=1}^T s_{jkt} y_{jkt} + \\
& \sum_{j=1}^J \sum_{\tau=1}^T \sum_{k=1}^K \sum_{t=1}^T (c_{jkt} + \sum_{l=\tau}^{t-1} h_{jl}) x_{j\tau kt}
\end{aligned}$$

subject to (P3)

$$p_{jt} - \sum_{k=1}^K \sum_{\tau=t}^T x_{j\tau kt} = 0 \quad \begin{matrix} j = 1, \dots, J, \\ t = 1, \dots, T, \end{matrix} \quad (8)$$

$$\sum_{j=1}^J \sum_{\tau=1}^t x_{j\tau kt} = d_{kt} \quad \begin{matrix} k = 1, \dots, K, \\ t = 1, \dots, T, \end{matrix} \quad (9)$$

$$p_{jt} \leq C_{jt} \quad \begin{matrix} j = 1, \dots, J, \\ t = 1, \dots, T, \end{matrix} \quad (10)$$

$$x_{j\tau kt} \leq \min(d_{kt}, U_{jkt}) y_{jkt} \quad \begin{matrix} j = 1, \dots, J, \\ k = 1, \dots, K, \\ t = 1, \dots, T, \\ \tau = 1, \dots, t, \end{matrix} \quad (11)$$

$$p_{jt}, x_{j\tau kt} \geq 0 \quad \begin{matrix} j = 1, \dots, J, \\ k = 1, \dots, K, \\ t = 1, \dots, T, \\ \tau = 1, \dots, t, \end{matrix} \quad (12)$$

The optimal cost of the LP relaxation of (P3) is greater than or equal to the optimal cost of the LP relaxation of (P2) since every solution to the LP relaxation of (P3) gives rise to a feasible solution to the LP relaxation of (P2). Figure 1.3 gives the network representation for the new formulation.

4. Computational Results

We test the two heuristic approaches on randomly generated problems. Size, structure, and the cost values seem to affect the difficulty of the problems. Therefore, we generate five groups of problems. Group 1 and Group 2 problems are single period problems, without inventory, and Groups 3, 4, and 5 are multi-period problems. To capture the effect of the structure of the network on the difficulty of the problems we vary the number of plants, retailers, and periods within each group. The problem sizes are given in Table 1. Problems 1 through 5 are in Group 1, 6 to 10 are in Group 2, 11 to 15 are in Group 3, 16 to 20 are in Group 4, and Group 5 consists of problems 21 to 25.

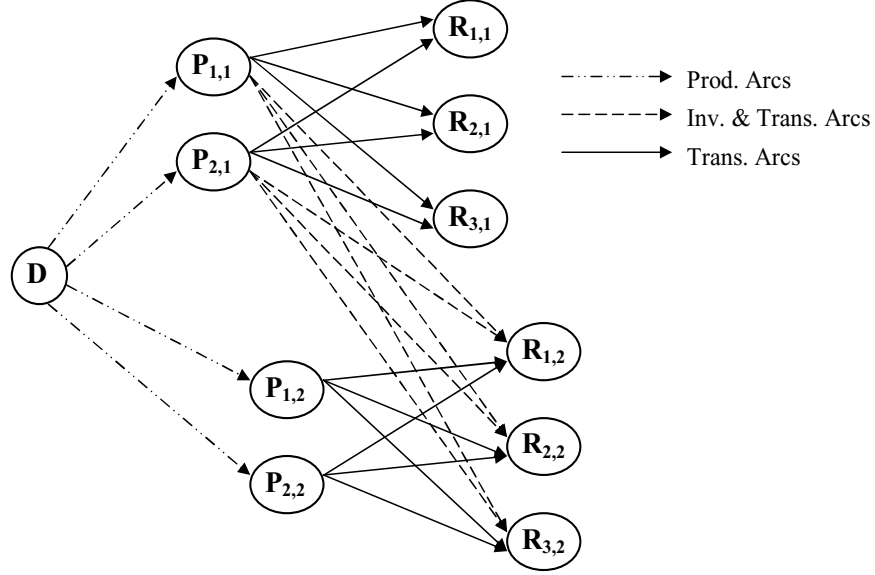


Figure 1.3. Extended network representation

Table 1. Problem Sizes

Problem	No. of Plants	No. of Retailers	No. of Periods	No. of Arcs
1	25	400	1	10025
2	50	400	1	20050
3	75	400	1	30075
4	100	400	1	40100
5	125	400	1	50125
6	125	200	1	25125
7	125	250	1	31375
8	125	300	1	37625
9	125	350	1	43875
10	125	400	1	50125
11	10	70	20	14390
12	15	70	20	21585
13	20	70	20	28780
14	25	70	20	35975
15	30	70	20	43170
16	30	60	5	9270
17	30	60	10	18570
18	30	60	15	27870
19	30	60	20	37170
20	30	60	25	46470
21	30	50	20	31170
22	30	55	20	34170
23	30	60	20	37170
24	30	65	20	40170
25	30	70	20	43170

It has been shown that the ratio of the total variable cost to the total fixed cost is an important measure of the problem difficulty [13]. Therefore, we randomly generate demands, fixed charges, and variable costs for each test problem from different uniform distributions as given in Table 2. Note that only the fixed charges are varied. The distances between the plants and the retailers are used as the unit transportation costs (c_{jkt}). For each plant and retailer we uniformly generate x y coordinates from a 10 by 10 grid and calculate the distances. The values of the parameter α , which is used in the construction phase of GRASP, were chosen from the interval (0, 1). Experiments indicated that better results were obtained for α in the interval [0.80, 0.85].

Table 2. Characteristics of Test Problems

Data set	s_{jt}, s_{jkt}	c_{jt}	h_{jt}	d_{kt}
A	5-15	5-15	1-3	5-55
B	50-100	5-15	1-3	5-55
C	100-200	5-15	1-3	5-55
D	200-400	5-15	1-3	5-55
E	1000-2000	5-15	1-3	5-55

For each problem size and for each data set we randomly generated 10 problem instances. The algorithms are programmed in the C language, and CPLEX 7.0 callable libraries are used to solve the linear NFPs and the MILPs. The algorithms are compiled and executed on an IBM computer with 2 Power3 PC processors, 200 Mhz CPUs each.

The errors found for problems using data set A were the smallest. This is expected since the fixed charges for this set are small. As the fixed charge increases the linear approximation is not a close approximation anymore. Thus, the heuristic approaches presented gave worse solutions for data set E. Nevertheless, all errors reported were less than 9%. All errors reported here are with respect to the lower bounds and are given by:

$$Error = \frac{BestUpperBound - LowerBound}{LowerBound} * 100$$

Table 3 reports the number of times the gap between the upper and lower bounds was zero for data set A (note that 50 problems were solved for each group). The number of times the heuristic approaches found the optimal solution may, however, be more than those numbers reported in the table since the errors are calculated with respect to a lower bound. For data set B, the errors were still small (all were under 0.6%). DSSP found errors equal to zero for 42 of the 50 problems in Group 1, for 36 of the problems in Group 2, and only for 1 problem in Group 3. All errors in Groups 4 and 5 were greater than zero. The number of errors equal to zero using GRASP, however, was 12 for Group 1 problems, only 1 for

Group 4 problems, and none for the other groups. As the fixed charges were increased the errors also increased.

Table 3. Number of times the error was zero for data set A

	DSSP	GR 8	GR 16	GR 32	GR 64	GR 128
Group 1	50	26	28	29	32	33
Group 2	50	19	23	27	30	32
Group 3	42	24	25	26	29	30
Group 4	30	17	22	29	30	32
Group 5	29	15	18	20	22	23

GR 8: GRASP with 8 iterations,

GR 16: GRASP with 16 iterations, etc.

The maximum error encountered for data set A set was 0.02% for both heuristics. The results show that DSSP and GRASP are both powerful heuristics for these problems, but another reason for getting such good solutions is because the problems in set A are relatively easy. We were actually able to find the optimal solutions from CPLEX for all problems in this set in a reasonable amount of time. On average, DSSP took less than a second to solve the largest problems in Groups 1 and 2. GRASP with 8 iterations took about 1.5 seconds for the same problems. Finding lower bounds also took about a second whereas it took more than 25 seconds to find optimal solutions for each of these problems. The problems in Groups 3, 4, and 5 were relatively more difficult. Average times were respectively 8, 10, 15, and 50 seconds for DSSP, GR 8, LP, and MILP. Optimal solutions were found for all problems using data set B as well. However, for data set C, optimal solutions were found for only Group 1, 2, and 3 problems and for some of the problems in Groups 4, and 5. For data set D, optimal solutions were found for some of the Group 1 and 2 problems. Finally, data set E was the most difficult set. CPLEX was able to provide optimal solutions for only a few of the Group 1 and 2 problems and none for the other 3 groups.

From Figure 1.4 we can see that DSSP gave better results for Group 1 and 2 problems and GRASP gave slightly better results for problems in Groups 3, 4, and 5. Similar behavior was observed for the other data sets. The results also indicate that GRASP is more robust in the sense that the quality of the solutions is not affected greatly by the network structure. The proposed local search also proved to be powerful. For example, average errors for the problems in Figure 1.4-d without the local search were roughly 2.9% for the DSSP and 11% for GR 8.

If more iterations are performed GRASP finds better solutions. Figure 1.5 shows that the errors decrease as the number of GRASP iterations increase. However, the computational time required by GRASP

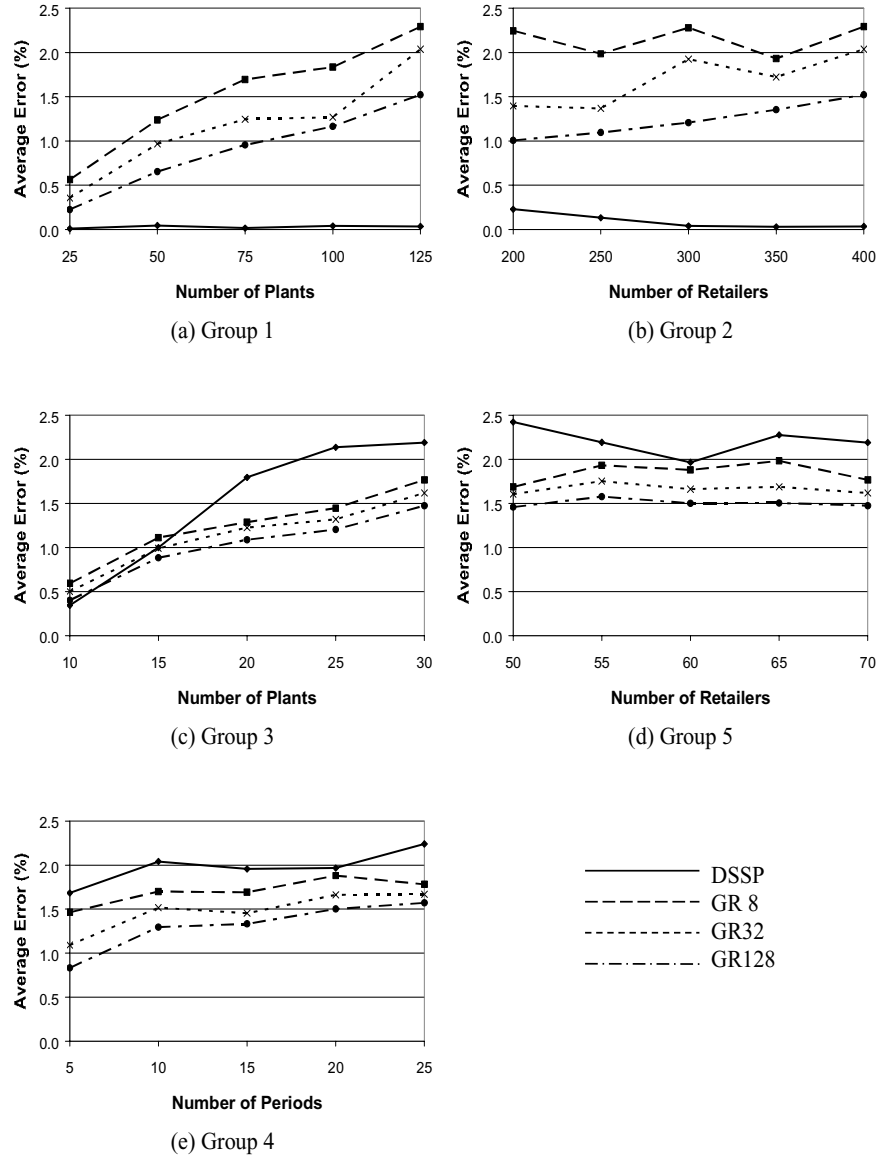


Figure 1.4. Average errors for problems using data set D

increases linearly as more iterations are performed. Most of the improvement is achieved earlier in the search. Performing more than 16 or 32 iterations does not seem to improve the solution quality by much. This suggests that the construction phase shows small diversification. To overcome this problem we modify our construction of solutions and try to diversify the solutions.

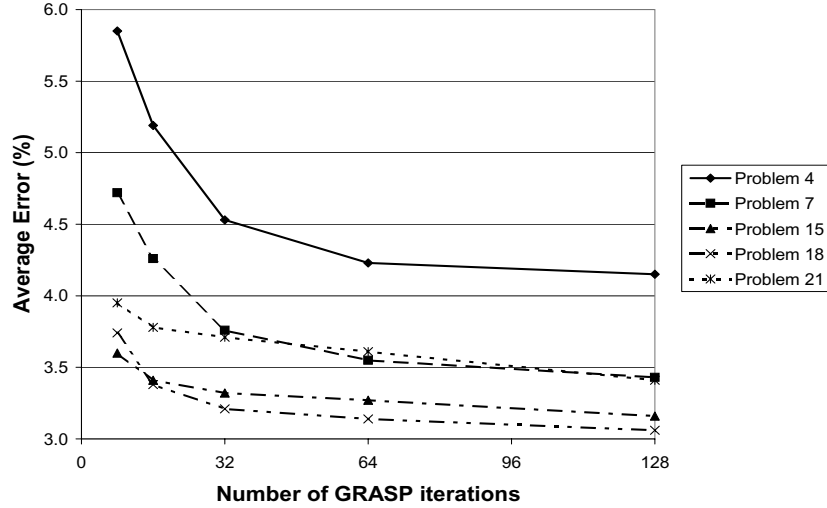


Figure 1.5. Number of GRASP iterations vs. solution quality

With the modified GRASP we try to generate solutions from different parts of the feasible region. This is done by allowing the construction of solutions that are not necessarily good. Our new criterion allows less desirable candidates to enter the restricted candidate list. We use $RCL := \{j : a \leq \lambda_j \leq a + b\}$ instead of $RCL := \{j : \lambda_j \leq \frac{\Lambda}{\alpha}, 0 < \alpha \leq 1\}$ where $a = \min\{\lambda_j : j = 1, \dots, J\}$ and $b = \alpha * (\max\{\lambda_j : j = 1, \dots, J\} - \min\{\lambda_j : j = 1, \dots, J\})$ initially. After a fixed number of solutions are generated using these initial values, a is changed ($a := a + b$) and another set of solutions is generated. If at any point RCL is empty, then the interval is enlarged and $RCL := \{j : a \leq \lambda_j \leq a + 2b\}$. Once a reaches $\max\{\lambda_j : j = 1, \dots, J\}$ it is set back to $\min\{\lambda_j : j = 1, \dots, J\}$. This process is done until a desired number of different solutions are generated.

The best results were obtained for α in the interval $[0.05, 0.20]$ with 16 solutions generated each time. Since GRASP did relatively poorly for Group 1 and 2 problems, we present results in Table 4 that show the improvement achieved with the new implementation of GRASP. To have a reasonable comparison, the new GRASP approach was run until 128

different solutions were investigated. As we can see there is significant improvement over the traditional GRASP approach. However, DSSP still seems to be doing better for these problems and also takes less time.

Table 4. Average errors for problems with data set E

Problem	DSSP		GR 128		New GR	
	Err.	CPU	Err.	CPU	Err.	CPU
1	0.02	0.32	3.86	4.14	0.82	4.55
2	0.17	0.78	4.11	8.05	1.95	8.81
3	0.24	1.14	3.55	11.47	1.72	13.12
4	0.54	1.75	4.15	15.03	2.02	17.26
5	0.56	2.85	3.86	19.08	2.00	22.17
6	2.83	2.12	2.91	9.19	2.45	10.69
7	1.75	2.17	3.43	11.79	2.18	13.57
8	1.23	2.04	4.03	14.04	2.29	16.40
9	0.86	2.32	3.58	16.28	2.04	19.03
10	0.56	2.92	3.86	19.01	2.00	22.17

Err: Average Error (%), CPU: CPU Time (sec)

5. Concluding Remarks

We have presented a DSSP and a GRASP algorithm for the uncapacitated concave production-transportation-inventory problem. Computational results for test problems of varying size, structure, and fixed charge costs are provided. An extended formulation is given. The linear programming relaxation of this formulation provides tight lower bounds. Errors are reported with respect to the lower bound and they were all less than 9%.

For Group 1 and 2 problems DSSP gave significantly better results, but for groups 3, 4, and 5 average errors obtained from GRASP were smaller. In terms of computational time, 8 to 16 iterations of GRASP was roughly equal to the time DSSP takes. As more iterations are performed, GRASP finds better solutions at the expense of added computational time.

DSSP seems to be more sensitive to problem structure and input data, but the errors obtained from GRASP were more robust. To improve the performance of our GRASP algorithm we implemented a modified approach to diversify the solutions constructed. This led to significant improvements, but for single period problems the errors obtained by DSSP were still smaller. GRASP can further be improved since it is a multi-start approach and is suitable for parallel implementation.

Acknowledgments

We would like to thank Sandra D. Eksioglu and Professor Joseph P. Geunes for their valuable and constructive comments, which have helped improve the quality of this paper.

References

- [1] Bell, G.J., Lamar, B.W., and Wallace, C.A. (1999), Capacity improvement, penalties, and the fixed charge transportation problem, *Naval Research Logistics* 46, 341-355.
- [2] Diaby, M. (1991), Successive linear approximation procedure for generalized fixed-charge transportation problem, *Journal of the Operational Research Society* 42, 991-1001.
- [3] Du, D.-Z., Pardalos, P.M. (eds.) (1993), *Network Optimization Problems*, World Scientific, Singapore.
- [4] Eksioglu, B., Eksioglu, S.D., and Pardalos, P.M. (2001), Solving Large Scale Fixed Charge Network Flow Problems, in: Daniele, P. and Maugeri, A. (eds.), *Variational Inequalities and Equilibrium Models*, Kluwer Academic Publishers.
- [5] Eksioglu, S.D., Pardalos, P.M., and Romeijn, H.E. (2002), A Dynamic Slope Scaling Procedure for the Fixed-Charge Cost Multi-Commodity Network Flow Problem, in: Pardalos, P.M. and Tsitsiringos, V.K. (eds.), *Financial Engineering, e-Commerce and Supply Chain*, Kluwer Academic Publishers.
- [6] Feo, T.A. and Resende, M.G.C. (1995), Greedy randomized adaptive search procedures, *Journal of Global Optimization* 6(2), 109-133.
- [7] Freling, R., Romeijn, H.E., Romero Morales, D., and Wagelmans, A.P.M. (1999), A branch-and-price algorithm for the multi-period single-sourcing problem, Research Report 99-12, Department of Industrial and Systems Engineering, University of Florida.
- [8] Geunes, J., Pardalos, P.M., and Romeijn, H.E. (eds.) (2002), *Supply chain management: models, applications, and research directions*, Kluwer Academic Publishers, The Netherlands.
- [9] Ghannadan, S., Migdalas, A., Tuy, H., and Varbrand, P. (1994), Heuristics based on tabu search and lagrangean relaxation for the concave production-transportation problem, *Studies in Regional and Urban Planning* 3, 127-140.

- [10] Guisewite, G.M. and Pardalos, P.M. (1990), Minimum concave-cost network flow problems: applications, complexity, and algorithms, *Annals of Operations Research* 25, 75-100.
- [11] Guisewite, G.M. and Pardalos, P.M. (1991), Algorithms for the single-source uncapacitated minimum concave cost network flow problem, *Journal of Global Optimization* 1, 245-265.
- [12] Guisewite, G.M. and Pardalos, P.M. (1993), A polynomial time solvable concave cost network flow problem, *Networks* 23, 143-147.
- [13] Hochbaum, D.S. and Segev, A. (1989), Analysis of a flow problem with fixed charges, *Networks* 19, 291-312.
- [14] Holmqvist, K., Migdalas, A., and Pardalos, P.M. (1998), A GRASP algorithm for the single source uncapacitated minimum concave-cost network flow problem, *DIMACS Series in Discrete Mathematics and Theoretical Computer Science* 40, 131-142.
- [15] Horst, R. and Pardalos, P.M. (eds.) (1995), *Handbook of Global Optimization*, Kluwer Academic Publishers, Dordrecht.
- [16] *ILOG CPLEX 7.0* (1999), ILOG, France.
- [17] Khang, D.B. and Fujiwara, O. (1991), Approximate solutions of capacitated fixed-charge minimum cost network flow problems, *Networks* 21, 689-704.
- [18] Kim, D. and Pardalos, P.M. (1999), A solution approach to the fixed charge network flow problem using a dynamic scaling procedure, *Operations Research Letters* 24, 195-203.
- [19] Kim, D. and Pardalos, P.M. (2000), Dynamic slope scaling and trust interval techniques for solving concave piecewise linear network flow problems, *Networks* 35, 216-222.
- [20] Kim, D. and Pardalos, P.M. (2000), A dynamic domain contraction algorithm for nonconvex piecewise linear network flow problems, *Journal of Global Optimization* 17, 225-234.
- [21] Klinz, B. and Tuy, H. (1993), Minimum concave-cost network flow problems with a single nonlinear arc cost, in: Du, D.-Z. and Pardalos, P.M. (eds.), *Network Optimization Problems*, World Scientific, 125-145.
- [22] Lamar, B.W. and Wallace, C.A. (1997), Revised-modified penalties for fixed charge transportation problems, *Management Science* 43, 1431-1436.
- [23] Larsson, T., Migdalas, A., and Ronnqvist, M. (1994), A Lagrangean heuristic for the capacitated concave minimum cost network flow problem, *European Journal of Operational Research* 78, 116-129.

- [24] McKeown, P.G. and Ragsdale, C.T. (1990), A computational study of using preprocessing and stronger formulations to solve general fixed charge problems, *Computers and Operations Research* 17, 9-16.
- [25] Pardalos, P.M. and Vavasis, S.A. (1992), Open questions in complexity theory for nonlinear optimization, *Mathematical Programming* 57, 337-339.
- [26] Romeijn, H.E. and Romero Morales, D. (1999), An asymptotically optimal greedy heuristic for the multi-period single-sourcing problem: the cyclic case, Research Report 99-11, Department of Industrial and Systems Engineering, University of Florida.
- [27] Romeijn, H.E. and Romero Morales, D. (1999), Asymptotic analysis of a greedy heuristic for the multi-period single-sourcing problem: the acyclic case, Research Report 99-13, Department of Industrial and Systems Engineering, University of Florida.
- [28] Romeijn, H.E. and Romero Morales, D. (2000), A greedy heuristic for a three-level multi-period single-sourcing problem, Research Report 2000-3, Department of Industrial and Systems Engineering, University of Florida.
- [29] Romeijn, H.E. and Romero Morales, D. (2001), A probabilistic analysis of the multi-period single-sourcing problem, *Discrete Applied Mathematics* 112, 301-328.
- [30] Sun, M., Aronson, J.E., McKeown, P.G., and Drinka, D. (1998), A tabu search heuristic for the fixed charge transportation problem, *European Journal of Operational Research* 106, 441-456.
- [31] Tuy, H. (2000), Strong polynomial solvability of a minimum concave cost network flow problem, *ACTA Mathematica Vietnamica* 25, 209-217.
- [32] Tuy, H. (2000), The MCCNFP problem with a fixed number of nonlinear arc costs: complexity and approximation, in: Pardalos, P.M. (ed.), *Approximation and Complexity in Numerical Optimization*, Kluwer Academic Publishers, 525-544.
- [33] Tuy, H., Dan, N.D., and Ghannadan, S. (1993), Strongly polynomial time algorithms for certain concave minimization problems on networks, *Operations Research Letters* 14, 99-109.
- [34] Tuy, H., Ghannadan, S., Migdalas, A., and Varbrand, P. (1995), The minimum concave cost flow problem with fixed number of nonlinear arc costs and sources, *Journal of Global Optimization* 6, 135-151.
- [35] Tuy, H., Ghannadan, S., Migdalas, A., and Varbrand, P. (1996), A strongly polynomial time algorithm for a concave production-

transportation problem with a fixed number of nonlinear variables, *Mathematical Programming* 72, 229-258.

- [36] Wu S.D. and Golbasi, H. (2002), Manufacturing planning over alternative facilities: modeling, analysis, and algorithms, in: Geunes, J., Pardalos, P.M., and Romeijn, H.E. (eds.), *Supply chain management: models, applications, and research directions*, Kluwer Academic Publishers, The Netherlands.